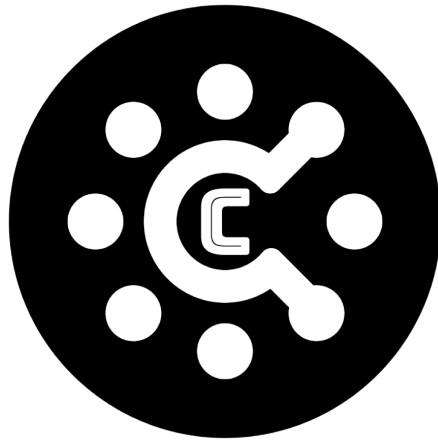




Cryptomask

Author : PV_Apps

Software Framework : Flutter

Prerequisites

To run Cryptomask application you need the below prerequisites to run/build the application. Xcode is required only if you need to run/build the application for the IOS platform.

- Windows/Mac OS/Ubuntu (Latest version)
- Flutter Version 3.16.8
- Dart Version 3.2.5 (stable)
- Android Studio Giraffe | 2022.3.1
- XCode Version 15.0.1 (15A507)
- VSCode Version 1.92.0 (Universal)

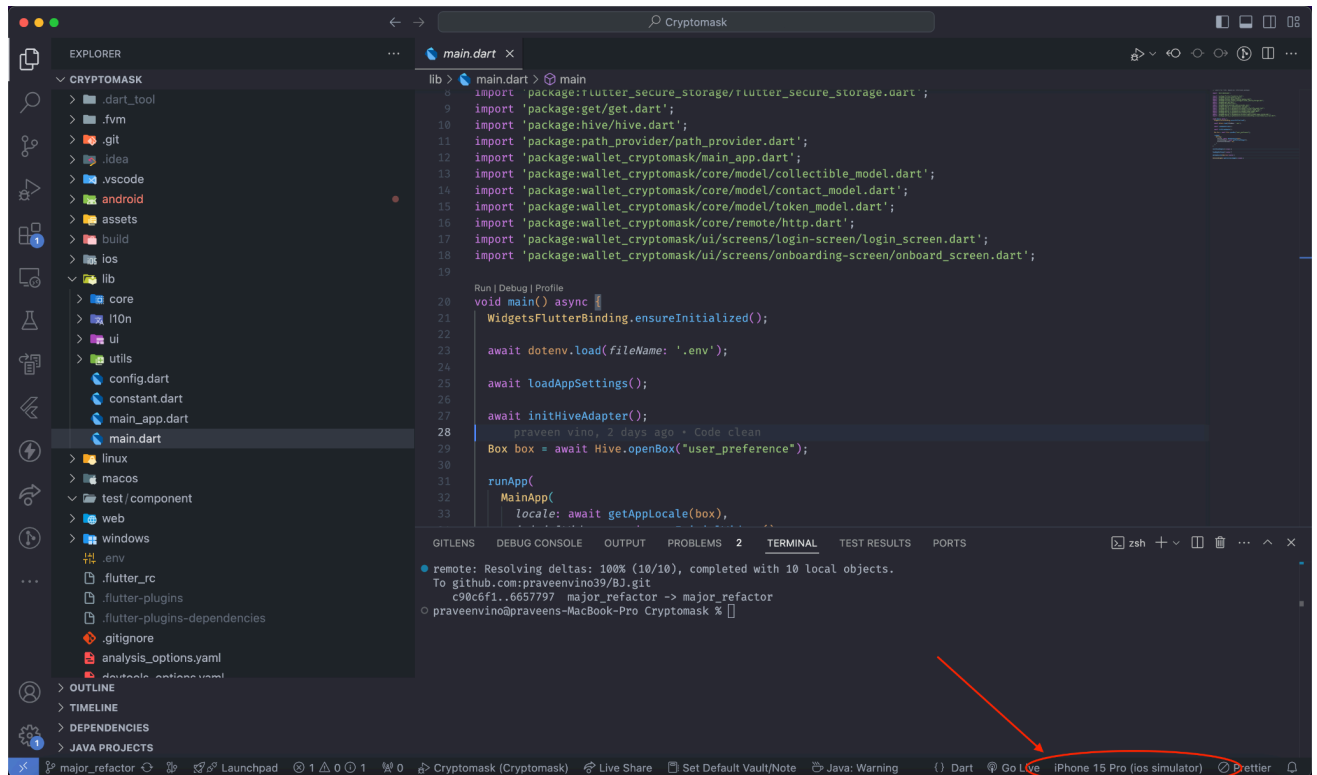
Quickstart

Running the application is very simple. You need RPC urls and Etherscan and Polygonscan API keys for Ethereum and Polygon network. You need to add those into the `.env` file (please create one if it doesn't exist).

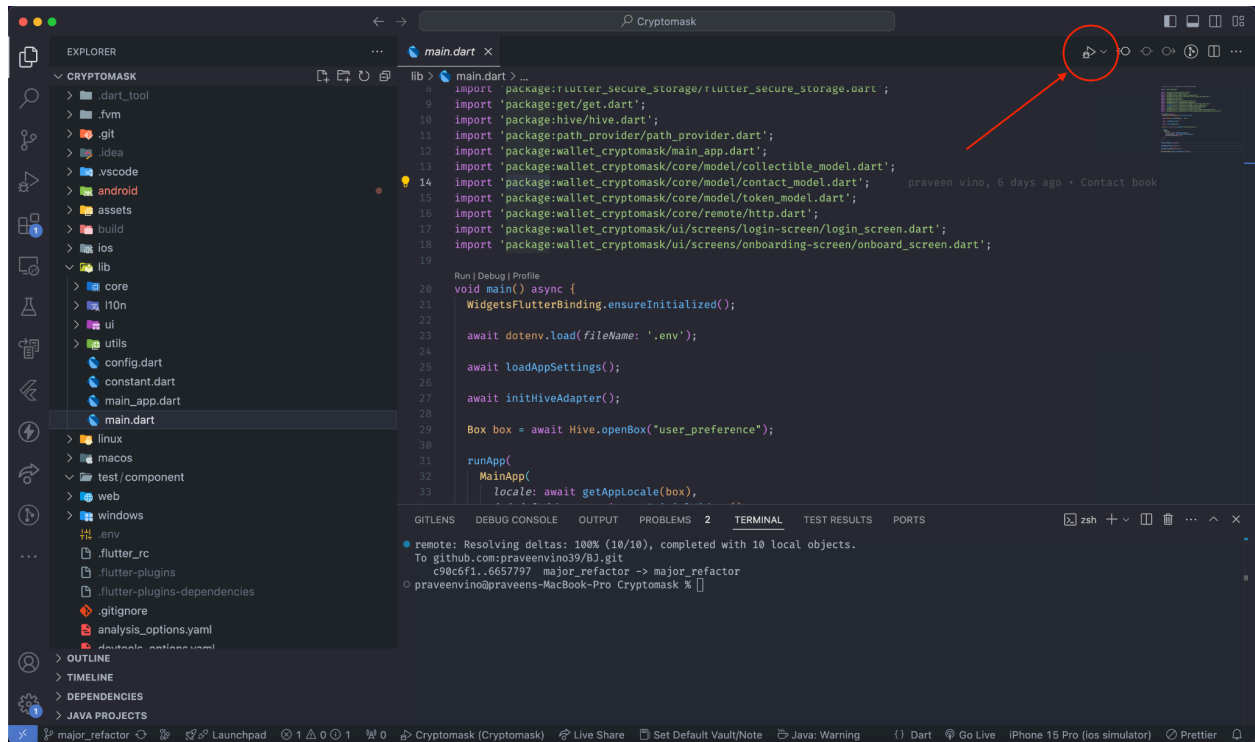
Example (Sample values)

```
ETHEREUM_RPC_URL=https://mainnet.infura.io/v3  
ETHEREUM_ETHERSCAN_API_KEY=R4UWZSAHBDVC95DACN4E7XVHMUXQ8ETIDG  
  
POLYGON_RPC_URL=https://polygon-rpc.com  
POLYGON_POLYSCAN_API_KEY=VNWSPE7JSBDSFA2HX1K7UKSPTN1CWD47
```

Open the project in VSCode and open the main.dart file from the explorer section on the left and click on the target selector and select targeted device and then click on the run button to run the application.



Selecting target



Running the application

Customisation

Changing the application name

You can easily name the application. All you need to do is install a global flutter dependency called [rename](#).

Activate plugin using following command

```
flutter pub global activate rename
```

Run the following command to rename the application

```
rename setAppName --targets ios,android --value  
"YourAppName"
```

This is rename application name, However in order to rename application name reference inside the app you need edit **appName** variable in the **config.dart** file.

Changing the Application ID and Bundle Identifier

If you want to publish your application to Appstore and Playstore you need to change your applicationId and Bundle Identifier before submitting the application for review.

Changing this is also a very easy task. There are a lot of methods to do this, But we're using the same [rename](#) plugin to do this.

Run the following command to change the applicationId and bundle Identifier for Android and ios respectively.

```
rename setBundleId --targets ios,android --value  
"com.example.bundleId"
```

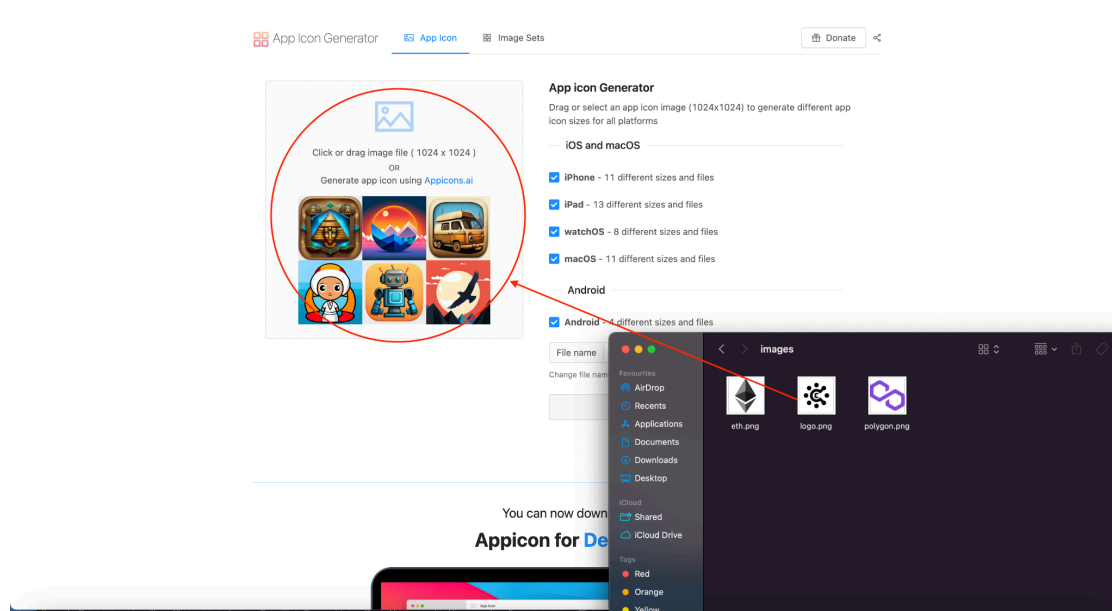
Replace `com.example.bundleId` with your identifier (Needs to be unique)

Changing App Icon and logo

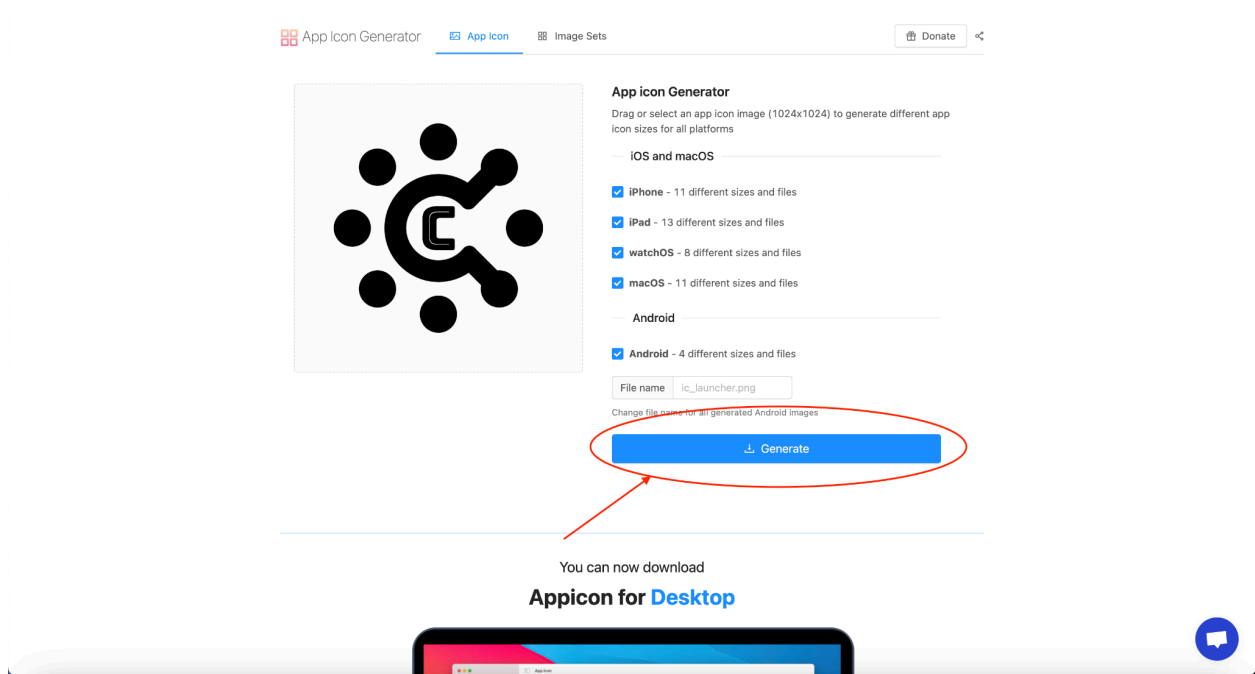
To change the App icon and logo you need to, replace your logo with logo.png inside `assets/images/` folder this will change the logo used inside the application, However to change the App icon you need to generate an icon set for Android and IOS.

Generating icon set for Android and IOS

Open [App Icon Generator](#) web app drag and drop your app icon and click on generate, It will download a .zip file containing your icon set. Extract the zip file and replace `AppIcon.appiconset` folder with `ios/Runner/Assets.xcassets/AppIcon.appiconset` folder and replace folders under android folder with folders inside `android/app/src/main/res`



Dropping logo png to App Icon Generator



Generating App Icon Set

Changing App theme color

Changing theme color is very simple, You just need to change the `kPrimaryColor` variable in `constant.dart` file.

Changing Browser homepage

Inorder to change the browser homepage, You just need to change the `BROWSER_HOMEPAGE` variable in the `.env` file.

Adding additional language

To add additional language, You need localized strings for the language to need to add ref `en` variable in `translation.dart` you need to add json like object for targeted language. Below `en` variable. And add entry for the targeted language in `supportedLocales` array inside `locale_provider.dart` and also add an addition if statement inside `loadTranslationsFromJson` function to support the targeted language.

Architecture

Cryptomask wallet uses Provider pattern throughout the application to maintain global states across the entire app lifecycle.

Important Classes

The **WalletProvider** class serves as the backbone of the Cryptomask application, enabling efficient and secure management of cryptocurrency wallets. This class is designed to handle multiple blockchain networks, manage wallet accounts, execute transactions, and provide seamless interaction with decentralized applications (dApps) through WalletConnect integration.

The **ContactProvider** class is an integral part of the Cryptomask application, responsible for managing user contacts associated with different blockchain networks. This class facilitates the storage, retrieval, and management of contact information within a Hive database, allowing users to easily maintain and interact with their contact lists.

The **CreateWalletProvider** class is a key component of the Cryptomask application, facilitating the secure creation and management of cryptocurrency wallets. It handles the generation of wallet passphrases, the creation of wallets with password protection, and the secure storage of wallet credentials using Flutter's **FlutterSecureStorage**. Additionally, it manages interactions with the application's backend for user registration and ensures that all operations are executed efficiently using Dart's isolate mechanism.

The **LocaleProvider** class is responsible for managing and persisting the language settings of the Cryptomask application. It allows users to switch between supported locales, ensuring that the application's interface adapts to the selected language. The provider interacts with Hive to store and retrieve the user's locale preference, ensuring that the application maintains the selected language across sessions.

The **NetworkProvider** class in the Cryptomask application is responsible for managing the blockchain networks that the application interacts with. It initializes a list of supported networks, which include essential information like RPC URLs, chain IDs, and API keys. This provider ensures that the application can connect to and interact with different blockchain networks, such as Ethereum and Polygon, by providing the necessary configuration details.

The **TokenProvider** class is designed to manage and interact with ERC-20 tokens within the Cryptomask application. It handles token data retrieval, balance checks, transactions, and token-specific operations, integrating with both local and remote services to ensure accurate and up-to-date information.

The **BrowserProvider** class manages the state and functionality of a web browser within the Cryptomask application. It handles various aspects of browser interactions, including navigation, mode settings, and data retrieval, ensuring a dynamic and responsive browsing experience.

The **SocketService** class handles real-time communication using WebSocket through the **socket_io_client** package. It provides methods for connecting to the WebSocket server, sending and receiving messages, and managing chat sessions.

The **MessageEngine** class manages messaging functionality within the application. It integrates with **SocketService** to handle real-time messaging and provides a set of methods for sending and receiving messages. This class uses the **ChangeNotifier** pattern to notify listeners of updates in the chat messages.

The **RemoteServer** class provides a set of static methods for interacting with remote APIs and services. It handles various operations related to user management, token information, transactions, and external API requests. This class serves as a bridge between the application and backend services or third-party APIs.

Building Release build (Android)

To build release version version of Android application you need run following commands

```
flutter build apk
```

You can test the application in release mode with this version, However you can't submit this build to Playstore publishment.

To build a Play Store publishable application you need to sign the application and generate **aab** file.

[Visit this Official Flutter Documentation](#) to sign the app

Once you sign the app, you can run the following command to generate **aab**.

```
flutter build appbundle
```

The release bundle for your app is created at **[project]/build/app/outputs/bundle/release/app.aab**.

Building Release build (IOS)

Before beginning the process of releasing your app, ensure that it meets [Apple's App Review Guidelines](#).

Please refer to this [Official Flutter Documentation page](#) for building an IOS application.